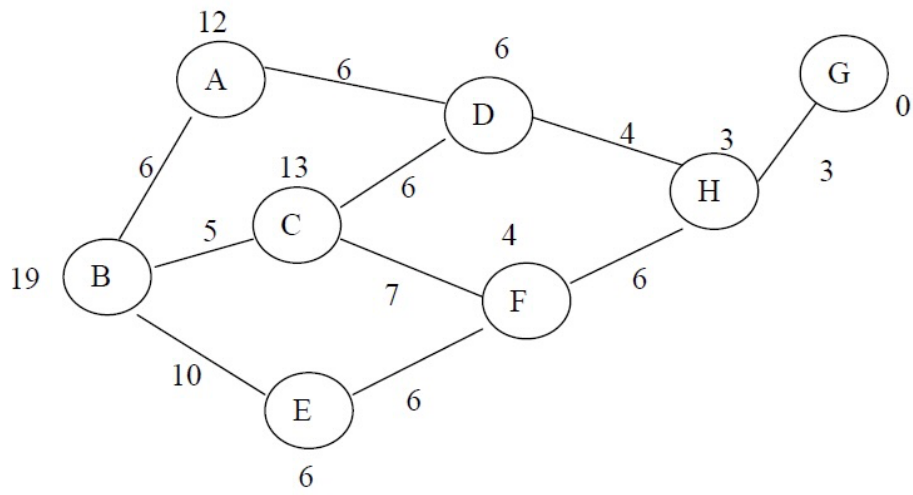


Mock - 2024

1.
- (a) Calculate the path that takes the shortest time to travel from the **B** node to the **G** node, using the A* algorithm.
Show your working in the table provided.



Start node	Destination node	Cost from start node (g)	Heuristic (h)	Total (f = g + h)
B	A	6	12	18
B	C	5	13	18
B	E	10	6	16
E	F	10+6=16	4	20
F	C	16+7=23	13	36
F	H	16+6=22	3	25
H	D	22+4=26	6	32
H	G	22+3=25	0	25

FINAL PATH = **B E F H G**

- [5]
- (b) Describe the purpose of both the A* algorithm and Dijkstra's algorithm.
 - The A* algorithm is used for finding the shortest path between a start and a goal node.
 - Dijkstra's algorithm finds the shortest path from a single source node to all other nodes in a weighted graph.
- [2]

Computer Science 9618

Paper 3 – Mock 2025

Marking Scheme

ZAK...

Mock - 2024

2. Write the assembly language program code that represents the following pseudocode algorithm.

```
COUNT ← 0
OUTPUT B
WHILE COUNT <> 5
    OUTPUT B
    COUNT ← COUNT + 1
ENDWHILE
OUTPUT A
END
```

Label	Op code	Operand	Comment
	LDM	#0	// load 0
	STO	COUNT	// store in COUNT
	LDD	B	// load B
	OUT		// output B
LOOP:	LDD	COUNT	// load COUNT
	CMP	#5	// is COUNT = 5 ?
	JPE	ENDLOOP	// jump to ENDLOOP if TRUE
	LDD	B	// load B
	OUT		// output B
	LDD	COUNT	// load COUNT
	INC	ACC	// increment ACC
	STO	COUNT	// store in COUNT
	JMP	LOOP	// jump to LOOP
ENDLOOP:	LDD	A	// load A
	OUT		// output A
	END		// end program

[4]

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

3.

(a) Given the following two code snippets, analyze the time complexity of each and explain your reasoning. Assign the appropriate Big O notation to represent the time complexity.

Code Snippet 1:

```
FOR i ← 1 TO n
  FOR j ← 1 TO n
    OUTPUT "Hello, ZAK!"
  NEXT
NEXT
```

Big O notation: $O(n^2)$

Brief explanation:

- Contains a nested loop structure.
The outer loop runs n times.
For every iteration of the outer loop, the inner loop also runs n times.
- Total iterations of the inner loop are $n \times n$, which is n^2 .
- The number of iterations increases quadratically with the input size n .
- Thus, the time complexity is quadratic and represented as $O(n^2)$.

Any 2

[3]

(b)

Code Snippet 2:

```
i ← n
WHILE i > 0
  OUTPUT "Hello, ZAK!"
  i ← i DIV 2
ENDWHILE
```

Big O notation: $O(\log n)$

Brief explanation:

- Consists of a single loop that decreases the value of i exponentially.
In each iteration, i is halved.
The loop continues until i becomes 0.
- The number of iterations is proportional to the logarithm of the starting value n .
- This division by 2 pattern results in a logarithmic time complexity.
- Hence, the time complexity is logarithmic, denoted as $O(\log n)$.

Any 2.

[3]

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

4. An international conference categorizes its submissions using codes generated according to specific rules. These rules are defined using the Backus-Naur Form (BNF), outlined as follows:

```
 <Category> ::= A|B|C|D|E
 <Year> ::= '20' <Digit><Digit>
 <Digit> ::= 0|1|2|3|4|5|6|7|8|9
 <SubmissionCode> ::= <Category><Year><Digit><Digit><Digit>
```

- (a) Create a definition for SubmissionCode as a record type, including components for Category, Year, and a sequence number.:

```
Record SubmissionCode
  DECLARE Category: (A, B, C, D, E)
  DECLARE Year: STRING // to hold '20' and two
                        // more digits.
  DECLARE SequenceNumber: INTEGER //a three-digit sequence
                                  // number, but represented as
                                  // an integer for simplicity.
End Record [2]
```

- (b) State whether each of the following is a valid code or an invalid code. In the case of an invalid code, explain how the BNF rules given in question 4. are broken.

- (i) A2023012 [2]

This code follows the BNF rule correctly, with A as a valid category, 2023 as a valid year, and 012 as a valid sequence number. Therefore, **it is a valid code.**

- (ii) C201875 [2]

This code doesn't appear to follow the BNF rule, with C as a valid category, 2018 as a valid year, and 75 as an invalid sequence number.

The confusion might arise with the length of the sequence number; however, according to the provided BNF:

```
<SubmissionCode> ::= <Category><Year><Digit><Digit><Digit>
```

a sequence number should have three digits, and 75 doesn't fulfill this requirement. Therefore, **it is an invalid code.**

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

- (c) The conference introduces a new code system to differentiate submissions. Alter and/or extend the BNF definition given in **4** above to:

Define `<ExtendedSubmissionCode>` that includes an **optional** abstract prefix "ABS-" before the `<Category>`.

[4]

Extension of BNF for optional abstract prefix "ABS-"

To extend the BNF to include an optional abstract prefix ABS- before the `<Category>`, we can introduce an optional non-terminal `<Prefix>` that either contains "ABS-" or nothing. The updated BNF definition would look like this:

```
<Prefix> ::= 'ABS-' | x
<Category> ::= A|B|C|D|E
<Year> ::= '20' <Digit><Digit>
<Digit> ::= 0|1|2|3|4|5|6|7|8|9
<ExtendedSubmissionCode> ::=
<Prefix><Category><Year><Digit><Digit><Digit>
```

Here, x denotes an empty string, allowing for the prefix to be optional. This definition enables the creation of submission codes both with and without the ABS- prefix before the category, accommodating the requirement for distinguishing abstract submissions.

----- **OR**

Another way to extend the BNF to include an optional abstract prefix ABS- before the `<Category>`, we can introduce optional non-terminals `<Prefix>` that contains ABS-, and `<X>` that contains `<Prefix> <Category>` or just `<Category>`. The updated BNF definition would look like this:

```
<Prefix> ::= 'ABS-'
<Category> ::= A|B|C|D|E
<X> ::= <Prefix><Category> | <Category>
<Year> ::= '20' <Digit><Digit>
<Digit> ::= 0|1|2|3|4|5|6|7|8|9
<ExtendedSubmissionCode> ::= <X><Year><Digit><Digit><Digit>
```

This definition enables the creation of submission codes both with and without the ABS- prefix before the category, accommodating the requirement for distinguishing abstract submissions.

----- **OR**

Since there might be other ways to write, alter, or extend the required BNF rules, any correct modification is acceptable.

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

5. The following statements pertain to a university's course enrollment system, described using a predicate logic based programming language where facts and rules state what to do but not how to do it.

```
01 completed(zak,math101,a) .
02 completed(mike,math101,c) .
03 completed(sara,math101,b) .
04 completed(alex,math101,d) .
05 hasLicence(zak) .
06 hasLicence(sara) .
07 passedTheoryTest(zak,compsci201) .
08 passedTheoryTest(sara,compsci201) .
09 interestedInCourse(zak,compsci201) .
10 interestedInCourse(mike,compsci201) .
11 interestedInCourse(sara,compsci201) .
12 interestedInCourse(alex,compsci201) .
13 canEnroll(X,C)
    IF interestedInCourse(X,C) AND hasLicence(X) AND
    passedTheoryTest(X,C) .
```

Given the facts and rules above:

- a. Give the name for this type of programming language. [1]

Declarative programming language.

- b. Give one example of a fact. [1]

A fact is a basic assertion that can be true or false within the context of the declarative language program. An example from the provided code is `completed(zak,math101,a)`. This fact states that "zak" has completed "math101" with a grade "A".

NB: Any correct example is acceptable from provided clauses.

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

c. Give one example of a rule.

[1]

A rule is a logical statement that describes a relationship between facts, typically involving conditional logic to infer new facts. An example from the text is the rule for `canEnroll(X, C)` which specifies the conditions under which someone is interested in a course, has a license, and has passed the theory test:

```
canEnroll(X,C) IF interestedInCourse(X,C) AND hasLicence(X) AND  
passedTheoryTest(X,C) .
```

----- OR

```
canEnroll(X,C) :- interestedInCourse(X,C) , hasLicence(X) ,  
passedTheoryTest(X,C) .
```

d. Give one example of a goal.

[1]

A goal is a query that the system tries to satisfy based on the facts and rules. An example of a goal could be finding out who can enroll in compsci201, which involves querying the system with the conditions provided in the rule for `canEnroll`. E.g: `canEnroll(X, compsci201)`.

NB: Any valid example but should be related to the question.

e. Give one example of an instantiation.

[1]

An instantiation is a specific instance where a variable in a rule or a fact is replaced with a concrete value. Given the rule for `canEnroll(X, C)`, an instantiation would be the application of specific individuals to this rule, such as determining if "zak" can enroll in compsci201 based on the conditions.

Goal: `canEnroll(X, compsci201)`.

Instantiation: `X = Zak, Sara`

NB: Any valid example but should be related to the question.

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

f. Create a rule that identifies students who are not eligible to enroll in a specific course because they:

-  either have not passed the required theory test for the course,
-  or do not have a license.

A rule for identifying students not eligible to enroll in a specific course could be defined by considering students who are interested in a course but either have not passed the required theory test for the course or do not have a license:

```
ineligibleForEnrollment(X,C)
IF interestedInCourse(X,C) AND
NOT (hasLicence(X) AND passedTheoryTest(X,C)) .
```

```
----- OR -----

ineligibleForEnrollment(X,C) :- interestedInCourse(X,C),
NOT (hasLicence(X), passedTheoryTest(X, C)) .
```

This rule checks if a student is interested in a course and then negates the conjunction of having a license and having passed the theory test, thus identifying students who fail to meet at least one of the enrollment criteria.

[3]

Computer Science 9618
Paper 3 – Mock 2025 **Marking Scheme**
ZAK...

Mock - 2024

6. In any calculations, you must show how you obtain your answers.
A binary number can also be written in a floating point representation as:

0	1	0	1	0	0	1	0
mantissa					exponent		

(a) (i) Write down the denary value represented by the **exponent** in this example. [1]

Answer: 2 (010 in binary is equal to 2 in denary)

(ii) Calculate the denary value of the binary number in this floating point representation. [2]

Answer:

01010 010

0.1010×2^2

$10.10 = +2.5$

----- OR

0.625×2^2

The mantissa is 0.01010 in binary, which is equal to $0.5 + 0.125 = 0.625$ in denary.

So, the denary value is $0.625 \times 4 = +2.5$

(b) Write, in binary, the largest number that can be represented in this floating point representation. [2]

Answer: mantissa: 01111, exponent: 011

In binary, the largest mantissa is all 1s, and the largest exponent is also all 1s. In this case, the largest mantissa is 0.1111 and the largest exponent is 7 (011 in binary). So, the largest number in this floating-point representation is $0.9375 \times 2^3 = 0.9375 \times 8 = 7.5$.

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

(c) Another binary number is written in the same format as:

0	0	1	1	1	0	1	1
mantissa					exponent		

Write this number in its normalised form.

[2]

Answer: mantissa: 01110, exponent: 010

The number can be normalized by shifting the mantissa one position to the left and decreasing the exponent by 1. This gives a mantissa of 0.1110 and an exponent of 010.

(d) State why binary numbers should be normalised.

[1]

Answer: Binary numbers should be normalized to maintain a consistent and efficient representation of floating-point numbers, which helps in reducing errors and simplifying arithmetic operations. Normalization ensures that there is only one representation for each distinct number, avoiding multiple representations for the same value. It also helps avoiding least significant fraction to drop from the mantissa.

7.

a. Explain why the number of machine cycles used by RISC and CISC to complete a single instruction is different.

[2]

RISC (Reduced Instruction Set Computer) architectures use simpler, fixed-length instructions that require fewer clock cycles to execute, while CISC (Complex Instruction Set Computer) architectures use more complex, variable-length instructions that can take multiple clock cycles to execute. RISC focuses on executing simple instructions quickly, whereas CISC allows for more powerful instructions that can perform multiple operations in a single instruction.

b. An array processor is used in SIMD systems.

(i) Explain the term SIMD processor.

[2]

SIMD (Single Instruction, Multiple Data) processor is a type of parallel processing architecture where a single instruction is applied to multiple data elements simultaneously. SIMD processors are designed to perform the same operation on

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

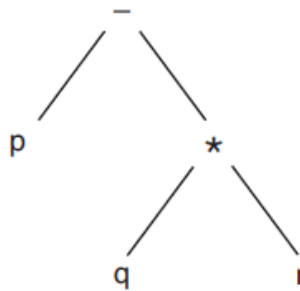
Mock - 2024

multiple data elements concurrently, which can significantly speed up tasks such as vector and matrix operations, image processing, and multimedia applications.

- (ii) Give one example of the type of computer used, for which a SIMD processor is most suitable. [1]

Answer: Graphics Processing Units (GPUs) or computers used for multimedia, video rendering, gaming consoles, and image processing.

- c. Following binary tree describes operands p , q and r with operators $-$ and $*$.



- (i) Write the reverse polish notation based on the binary tree above. [2]

Answer: $pqr*-$ (reverse polish notation)

- (ii) State the type of traversal that should be used to obtain the reverse polish notation from a binary tree. [1]

Answer: Postorder traversal / Left, Right, Root

- (iii) An expression in reverse polish notation is stated as: $t \ u \ * \ v \ +$
Showing how you obtained your answer, evaluate this expression, where:

$$T = 2, u = 3 \text{ and } v = 10$$

[2]

1. $2 \ 3 \ * \ 10 \ +$
2. $6 \ 10 \ +$
3. 16

Computer Science 9618

Paper 3 – Mock 2025

Marking Scheme

ZAK...

Mock - 2024

(iv) State data structure, other than binary tree, you used to evaluate RPN in c(iii). [1]

Stack [data structure]

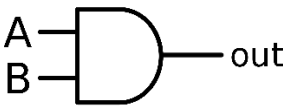
(v) State a symbol, often used for infix mathematical notations, that is not required when working with RPN. [1]

Parenthesis, ()

8. Complete truth table below for AND gate and draw the symbol for the same. [2]

(a)

Input A	Input B	Input C
0	0	0
0	1	0
1	0	0
1	1	1



(b) Table below list six Boolean laws. Three of them are correct. Shade the circles next to the three equations that are correct. [3]

Equation	Correct? (Shade three)
$A \cdot \overline{A} = 1$	☐
$A + B = \overline{\overline{A} \cdot \overline{B}}$	☒
$A + 1 = 1$	☒
$A \cdot (A + B) = A$	☒
$A + (A \cdot B) = B$	☐
$A \cdot 1 = 1$	☐

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

(c) Using the laws describing Boolean algebra,
Simplify the following Boolean expression.

[3]

$$(X + Y) \cdot (X + \overline{Y})$$

Answer: X

Solutions:

Option 1:

Given expression: $(X + Y) \cdot (X + Y')$

Applying the distributive law:

$$= (X \cdot X) + (X \cdot Y') + (Y \cdot X) + (Y \cdot Y')$$

Applying the idempotent law ($A \cdot A = A$) and the complement law ($A \cdot A' = 0$):

$$= X + (X \cdot Y') + (Y \cdot X) + 0$$

Now, notice that $(Y \cdot X) = (X \cdot Y)$, so we can rewrite the expression as:

$$= X + (X \cdot Y') + (X \cdot Y)$$

Now, we can apply the distributive law in reverse:

$$= X + X \cdot (Y' + Y)$$

Applying the complementary law ($A + A' = 1$):

$$= X + X \cdot 1$$

Applying the identity law ($A \cdot 1 = A$):

$$= X$$

Option 2:

Given expression: $(X + Y) \cdot (X + Y')$

Applying the distributive law:

$$= (X \cdot X) + (X \cdot Y') + (Y \cdot X) + (Y \cdot Y')$$

Applying the idempotent law ($A \cdot A = A$) and the complement law ($A \cdot A' = 0$):

$$= X + (X \cdot Y') + (Y \cdot X) + 0$$

Since $X \cdot Y'$ and $Y \cdot X$ are equivalent:

$$= X + (X \cdot Y') + (X \cdot Y')$$

Applying the idempotent law again:

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

$$= X + (X \cdot Y')$$

Now, applying the absorption law ($A + A \cdot B = A$):
 $= X$

Option 3:

$$(X + Y) \cdot (X + Y')$$

Applying the distributive law:
 $= X \cdot (X + Y') + Y \cdot (X + Y')$

Applying the absorption law ($A \cdot (A + B) = A$):
 $= X + Y \cdot (X + Y')$

Now, applying the consensus theorem ($(A + B)(A' + C)(B + C) = (A + B)(A' + C)$):
 $= X + Y \cdot X'$

Since $X \cdot X' = 0$:
 $= X + 0$
 $= X$

9. The TCP/IP protocol suite can be viewed as a stack of layers.

a. (i) Complete the stack by inserting the names of the missing layers below.

Application
Transport
Internet [Network Access]
[Data] Link

[3]

(ii) State how each layer of the stack is implemented.

[1]

By communicating with layers above and below.

OR

Each layer of the stack is implemented in software, with the exception of the (Data) Link layer, which is implemented in both software and hardware (network interface cards and physical network connections).

Computer Science 9618

Paper 3 – Mock 2025 Marking Scheme

ZAK...

Mock - 2024

b. A computer is currently running two processes:

-  Process 1 is downloading and opening a web page in browser software.
-  Process 2 is downloading an email using email client software.

Describe two tasks that the Transport layer performs to ensure that the incoming data is downloaded correctly. [4]

1. **Segmentation and Reassembly** – The Transport layer breaks incoming data into smaller packets and ensures they are reassembled correctly at the destination.

2. **Error Detection and Correction** – It checks for errors in received packets and requests retransmission if errors are detected.

c. Name a possible protocol that will be used by the Process 1. [1]

Hypertext Transfer Protocol (HTTP) or Hypertext Transfer Protocol Secure (HTTPS)

d. Name a protocol that will be used by Process 2. [1]

Post Office Protocol 3 (POP3) or Internet Message Access Protocol (IMAP)